

RISC-V-Based Computer Architectures: A Survey of Extensions, Accelerators, and Applications

Md Shadman Shakib Fuad, Md Junied Hossain, Md Nahid Mahmud, Md Saiduzzaman

Department of Computer Science and Engineering

ABSTRACT RISC-V has grown from a research instruction-set architecture (ISA) developed at UC Berkeley into a broad, industrially adopted open ecosystem spanning microcontrollers, vector accelerators, and heterogeneous systems-on-chip (SoCs). This survey reviews twenty papers, published mostly between 2022 and 2025, that extend, accelerate, or deploy RISC-V processors, and organizes them into three themes: (i) instruction-set extensions — standard (M, A, F, D, C), the Vector (V) extension, security/cryptographic extensions, and custom domain-specific instructions; (ii) hardware accelerators built around a RISC-V control core for AI/ML, digital signal processing (DSP), cryptography, and general coprocessor integration; and (iii) application domains, including embedded/IoT nodes, edge AI and wearables, automotive/real-time systems, and high-performance computing (HPC). Three trends recur across the literature. First, the Vector extension and its embedded variant (Zve32x) are the most broadly reused acceleration mechanism, with reported speedups ranging from roughly $2\times$ to over $100\times$ depending on workload. Second, most custom extensions are validated only at the FPGA or cycle-accurate-simulator level, with very few reaching taped-out silicon. Third, energy, area, and accuracy metrics are reported inconsistently, which complicates direct comparison. We summarize these findings in three comparative tables and two figures, and close by identifying open gaps — most notably the lack of standardized benchmarks for custom RISC-V accelerators and the limited security evaluation of the extensions themselves.

INDEX TERMS *accelerators, custom instructions, edge AI, embedded systems, instruction-set extensions, RISC-V, secure processors, vector extension*

I. INTRODUCTION

RISC-V is an open, royalty-free instruction set architecture (ISA) originally developed at the University of California, Berkeley. Unlike proprietary ISAs such as x86 and ARM, RISC-V is governed by an open standard maintained by RISC-V International, which allows academic groups, startups, and large semiconductor companies to design and modify processor cores without licensing fees. The base ISA (RV32I, RV64I, and RV128I) is deliberately minimal, and additional functionality is added through a modular set of standard and custom extensions [19], [20].

This modularity has made RISC-V attractive for a very wide range of use cases — from tiny embedded microcontrollers to vector-processing accelerators for machine learning and even experimental data-center processors. Over the past few years, the number of published extensions, accelerator designs, and application-specific RISC-V implementations has grown rapidly, which motivates the need for an up-to-date survey.

Several existing RISC-V surveys focus narrowly on either the software ecosystem or a single class of extension, and most pre-date the recent wave of Vector-extension-based AI accelerators and dedicated security extensions published between 2022 and 2025. Mezger et al. [19] reviewed the state of RISC-V software tooling as of 2022, while Cui et al. [20] cataloged official and custom ISA extensions up to 2023; neither captures the more recent generation of Vector-extension AI accelerators, hyperdimensional-computing coprocessors, or enclave-style security extensions surveyed here. This motivates an updated, application-oriented view that connects extensions, accelerators, and real deployments in one place.

It is also worth situating this survey's scope precisely: we do not attempt to cover RISC-V compiler theory, formal verification of RISC-V cores, or the RISC-V standardization process itself in any depth, since none of these are the primary focus of the twenty papers we selected. Readers interested specifically in the software-toolchain side of RISC-V are better served by Mezger et

al.'s dedicated survey [19], which we cite throughout but do not attempt to duplicate.

The contributions of this survey are as follows:

- 1) Reviewed the evolution and current state of standard and custom RISC-V extensions.
- 2) Classified RISC-V-based hardware accelerators by application domain (AI/ML, DSP, cryptography, custom coprocessors).
- 3) Summarized real-world and research applications of RISC-V across embedded, edge, automotive, and HPC domains.
- 4) Identified open research gaps and suggested directions for future RISC-V research.

The rest of this paper is organized as follows. Section II gives background on the RISC-V ISA. Section III describes our survey methodology. Section IV reviews RISC-V extensions. Section V discusses hardware accelerators built on RISC-V. Section VI covers application domains. Section VII presents a comparative discussion. Section VIII gives design recommendations for practitioners. Section IX outlines research gaps and future directions. Section X discusses limitations of this survey, and Section XI concludes the paper.

II. BACKGROUND ON THE RISC-V ISA

RISC-V follows the reduced instruction set computer (RISC) design philosophy: a small set of simple, fixed-length instructions that can be efficiently pipelined and implemented in hardware. The base integer ISA comes in three widths — RV32I (32-bit), RV64I (64-bit), and RV128I (128-bit, largely experimental) — and defines only the essential instructions needed for a functioning processor, such as arithmetic, logic, control flow, and memory access. Additional capability is added through extensions, each identified by a letter or short code. The most common standard extensions are M (integer multiplication and division), A (atomic memory operations), F and D (single- and double-precision floating point), and C (compressed 16-bit instructions for code-size reduction). A processor implementing all of these is commonly referred to as RV32GC or RV64GC, where 'G' is shorthand for the IMAFD combination.

Beyond the user-level integer ISA, RISC-V defines a separate privileged architecture that specifies machine, supervisor, and user privilege levels, control-and-status registers, and a page-based virtual-memory scheme; this separation is what allows the same base ISA to scale from a bare-metal microcontroller core to a Linux-capable application processor. Several surveyed systems make explicit use of this layering: the CVA6 host core used in HULK-V [5] and in the Shaheen SoC [4] is a 64-bit, Linux-capable RISC-V core, while the security extension proposed by Schrammel et al. [11] is implemented as an addition to the privileged architecture's page-table-based memory management rather than as a new user-level instruction.

The open governance of RISC-V also explains the diversity of implementations covered in this survey: academic soft cores such as PicoRV32, Klessydra, and CVA6 sit alongside commercial silicon, and Sharma et al. [6] note that this openness is precisely what lets small IoT-chip teams avoid the licensing costs associated with ARM or x86 cores. Cui et al. [20] describe RISC-V as mature in the embedded-processor space but still emerging in cloud and high-performance computing, a characterization that is consistent with the mix of embedded and early-stage HPC papers reviewed in Section VI of this survey.

A further architectural point relevant to later sections is how extensions are actually added to a base core in practice. The surveyed papers reveal at least three distinct integration styles: extending the execution stage of an existing pipeline with a new functional unit, as in the HDCU [3] and the sparsity-aware CFUs of [16]; adding a wholly separate coprocessor cluster alongside a host core, as in Shaheen [4], HULK-V [5], and ColibriES [15]; and modifying the privileged/memory-management layer rather than the datapath at all, as in SPEAR-V [11]. These are not interchangeable choices — execution-stage extensions tend to report the largest per-operation speedups but the tightest coupling to a specific core's pipeline, while coprocessor-cluster designs are more portable across host cores but carry

higher communication overhead, a trade-off we revisit quantitatively in Section VII.

The twenty papers also differ in how explicitly they discuss RISC-V's relationship to other open and proprietary ISAs. ARM is mentioned as a cost or licensing baseline in [6] and as a source of ported software (CMSIS-NN) in [9]; x86 appears only as a general point of contrast in the background material of [19] and [20]; and no paper in our set directly benchmarks a RISC-V design against an ARM or x86 implementation of the same accelerator, meaning cross-ISA performance comparisons in this survey are necessarily indirect, drawn from separately reported baselines rather than head-to-head measurements on identical workloads.

III. SURVEY METHODOLOGY

This survey was produced by a four-member team using Google Scholar as the primary literature source, following the search protocol required for the course assignment. We searched combinations of the terms “RISC-V”, “extension”, “accelerator”, “vector extension”, and “custom instruction” together with application-specific terms such as “IoT”, “neural network”, and “enclave”, restricting results to papers published between 2022 and 2026 (one earlier work [6], posted to SSRN in 2022, was retained because it directly compares RISC-V cores for IoT and is still actively cited).

Each of the four team members read and logged at least four papers into a shared spreadsheet capturing the problem addressed, proposed method, key quantitative results, evaluation platform, and authors' stated limitations for every paper; this spreadsheet is the primary data source synthesized into the tables and comparative discussion of Sections IV–VII. In total, twenty papers were reviewed, exceeding the assignment's minimum of sixteen, spanning IEEE

journals and conferences, ACM conferences, arXiv/TechRxiv preprints, and one SSRN preprint.

We grouped the twenty papers into three non-exclusive themes for presentation: instruction-set extensions (Section IV), hardware accelerators (Section V), and application deployments (Section VI); several papers — for example the Shaheen SoC [4] and REALISE-IoT [1] — appear in more than one theme because they simultaneously propose an extension, an accelerator, and a target application. Papers that are themselves surveys ([6], [17], [19], [20]) are used throughout as secondary sources to contextualize the primary experimental papers rather than being confined to a single section.

Relative to the four survey papers already in our set, this work differs in scope rather than in depth: Mezger et al. [19] and Cui et al. [20] each systematically cover a single axis — software tooling and ISA-extension taxonomy, respectively — across a much larger literature base than the sixteen primary experimental papers we read in full, while Sharma et al. [6] and Reuther et al. [17] are themselves narrower comparisons (four IoT cores; commercial AI-accelerator vendor data) rather than broad literature reviews. Our contribution is therefore not a larger-scale systematic review but a synthesis that connects a specific, hand-selected set of 2022–2025 primary papers across the extension/accelerator/application boundary, cross-checked against the four existing surveys for context, which is the structure we found most useful for a course-length survey with a hard page and paper-count target.

Table 0 summarizes how the twenty papers were divided among the four team members for first-pass reading and note-taking; every paper was subsequently discussed by the full team before being placed into the extension/accelerator/application sections of this survey, so no single subsection reflects only one member's reading.

TABLE 0. Paper assignment by team member for first-pass review.

Team Member	Papers Reviewed	Short Titles	Count
Junied Hossain	[1]–[5]	REALISE-IoT, FFT/RVV, HDCU, Shaheen SoC, HULK-V	5
Shadman Shakib Fuad	[6]–[10]	RISC-V IoT Survey, GFDM/RVV, Tensor Opt., muRISCV-NN, CIDRE	5
Nahid Mahmud	[11]–[15]	SPEAR-V, Longnail, LLVM/ANN Extensions, RV-LGN, ColibriES	5
Saiduzzaman	[16]–[20]	Sparse-DNN CFU, AI/ML Accel. Survey, IndexMAC, Mezger Survey, Cui Survey	5

IV. RISC-V EXTENSIONS

Beyond the base integer ISA, RISC-V's extensibility is one of its defining strengths. This section reviews the major categories of extensions covered in the surveyed literature.

A. Standard Extensions (M, A, F, D, C)

Only one of the surveyed papers focuses primarily on the baseline standard extensions, but several others build directly on top of them. Sharma et al. [6] compare four open-source RV32IM cores — PicoRV32, MicroRV, RavenSoC, and an Ultraembedded RISC-V core — all of which implement only the base integer ISA plus the M extension for multiplication and division, and report clock frequencies ranging from 28.61 MHz for MicroRV on a Lattice HX8K FPGA up to 769 MHz for PicoRV32 on a larger Xilinx device, illustrating how much implementation choice, rather than ISA feature selection alone, affects real performance.

The compressed-instruction extension (C), which packs common instructions into 16 bits to shrink code size, and the atomic extension (A), which provides multicore synchronization primitives, are not the primary subject of any paper in our set but are assumed as part of the standard RV32GC/RV64GC baseline against which several accelerator papers report their speedups, for instance the CVA6 host cores used in HULK-V [5] and Shaheen [4]. Floating-point support (F/D) is likewise treated as a baseline capability rather than a research contribution in these papers, used mainly to run the reference software implementations that proposed hardware extensions are compared against.

B. Vector Extension (V) and Bit-Manipulation Extension (B)

The Vector extension is the single most reused mechanism among the surveyed extension papers. Vizcaino et al. [2] implement four RADIX-2 FFT variants using the RISC-V “V” extension on an Avispado core with a Vitruvius vector processing unit, reporting up to $16\times$ speedup on RISC-V hardware and up to $50\times$ over scalar execution for large transforms, though they note about 25% extra floating-point cycles for the largest FFT sizes. Razilov et al. [7] target 6G baseband signal processing, implementing GFDM modulation on the open-source Ara vector processor and reaching $60\times$ speedup over a scalar baseline, with throughput up to 784 MSymbols/s at 16 lanes — close to, but still below, a fixed-function vector DSP's 889 MSymbols/s.

For AI workloads specifically, Peccia et al. [8] integrate RVV tensor intrinsics into TVM's MetaSchedule autotuner, obtaining a 46% average latency improvement over GCC autovectorization and a 35% speedup over LLVM autovectorization on a commercial Banana Pi BPI-F3 board, while van Kempen et al. [9] hand-port ARM CMSIS-NN kernels to the embedded Zve32x vector subset, reaching $2.9\times$ – $12.2\times$ speedup over reference libraries — though this hand-tuning must be redone whenever the vector length changes, unlike the autotuned approach in [8]. Titopoulos et al. [18] instead add a single custom vector instruction, IndexMAC, for structured-sparse matrix multiplication in a decoupled RV64GC vector processor, achieving $1.80\times$ – $2.14\times$ speedup over a strong vectorized baseline at negligible extra hardware cost.

None of the surveyed papers focuses primarily on the Bit-Manipulation (B) extension in isolation; it appears only as background context in the extension-taxonomy surveys [19], [20], which note that bit-manipulation instructions are increasingly used for cryptographic and hashing workloads. This is a visible gap relative to the depth of coverage the Vector extension receives (three dedicated papers plus a fourth touching on it), suggesting bit-manipulation work is comparatively under-represented in the recent literature available to us.

A further observation from Table 1 is that the four Vector-extension papers cover four different points on the same design axis — vector length, lane count, and register-file layout — rather than four unrelated problems: [2] optimizes twiddle-factor and memory-access patterns for long vectors, [7] optimizes lane utilization and data layout, [8] optimizes autotuning across different vector-length configurations, and [18] adds a single new vector instruction to reduce redundant loads. Read together, they suggest that Vector-extension research has matured past simply 'porting a kernel to RVV' and is now focused on the harder second-order problems of memory movement, autotuning, and instruction-level refinement.

C. Security and Cryptographic Extensions

Two papers explicitly target hardware security. Schrammel et al. [11] propose SPEAR-V, a page-granular memory-tagging extension to the privileged architecture that isolates enclaves using a hardware tag store and a software Security Monitor, reporting essentially zero overhead on unprotected applications and about 1% overhead on protected ones on the Embench suite — a much lower metadata cost (24 bits per page) than the 128-bit metadata used by comparable commercial designs such as Intel TDX. Mao et al. [1] take a different angle, extending LoRaWAN IoT communication with a two-level public-key protocol (ECDH key exchange, EdDSA authentication) implemented as a lightweight secure RISC-V processor with hardware acceleration, achieving $5.6\times$ – $144.7\times$ speedup and $2.4\times$ – $12.3\times$ memory reduction over software cryptography on an FPGA-based IoT platform.

These two papers address complementary threat models: SPEAR-V isolates untrusted code from a trusted host, while REALISE-IoT accelerates trusted cryptographic primitives for a resource-constrained device, and neither considers physical side-channel attacks — SPEAR-V explicitly excludes bus-snooping and malicious-DRAM attacks from its threat model. The extension-taxonomy survey by Cui et al. [20] situates both works within a broader trend toward “Zk”-style scalar cryptography extensions and post-quantum-ready modular-reduction instructions, which none of our primary papers implement directly but which the survey flags as an active research direction.

Compared to the Vector-extension papers, the two security extensions in our set are much more conservative in their claimed performance gains — both aim for near-zero overhead rather than a large speedup — which reflects the different objective of a security extension: it must be adopted alongside existing, unmodified software with minimal disruption, whereas a Vector or custom-instruction extension is typically adopted specifically because a workload is being redesigned to use it. This difference in objective is useful context for the research gaps identified in Section VIII, since it explains why security-extension papers are harder to compare on a single 'speedup' axis than the accelerator papers in Section V.

D. Custom and Domain-Specific Extensions

The largest and most heterogeneous group of extension papers proposes entirely new, domain-specific instructions. Martino et al. [3] add a Hyperdimensional Computing Unit (HDCU) to the execution stage of a Klessydra-T03 core via a custom instruction-set extension, reporting extreme micro-benchmark speedups (up to $3844\times$ for associative search, though far more modest gains on complete real datasets such as CARDIO and EMG) and a clear area/frequency trade-off: LUT usage rises from about 1,030 to over 34,000 as SIMD width scales from 32 to 1024, while maximum frequency drops from 234 MHz to 140 MHz.

Oppermann et al. [12] take a tooling-first approach with Longnail, a high-level-synthesis flow that compiles a

portable, C-like hardware description language (CoreDSL) directly to SystemVerilog custom instructions, validated across four different open-source RISC-V cores and eight benchmark instruction extensions on a 22 nm ASIC flow — explicitly aiming for the vendor-portability that proprietary flows such as Tensilica TIE lack. Rezunov et al. [10] instead fully automate instruction discovery: their CIDRE flow disassembles a compiled RISC-V binary, builds a dataflow graph, enumerates candidate instruction fusions, and selects them with a greedy algorithm, reaching up to $2.47\times$ speedup on an AES benchmark with an average area increase of only 11.3%.

Two further papers target neural-network inference specifically at the instruction level. Kalyanasundaram Balasubramanian et al. [13] use an LLVM/Glow compiler-profiling flow to identify which operations in int8-quantized networks (a single perceptron, ResNet-18, VGG-11, LeNet-5) dominate runtime on a plain RV32I core, then propose targeted custom instructions that reach up to $13\times$ speedup in a Python-based cycle emulator, though no physical hardware implementation yet exists. Wang et al. [14] instead target Logic Gate Networks, adding four custom instructions for encoding and decoding real-valued data into the Boolean bitstreams these networks operate on, cutting encoding/decoding runtime by 79–94% depending on bit-width, although the resulting network’s CIFAR-10 accuracy (51.3%) trails comparable CNN/SNN RISC-V accelerators from prior work (78.9–88.5%), a trade-off the authors acknowledge openly.

From a practitioner’s standpoint, the standard extensions are best understood as a checklist rather than a research

target: [6]’s comparison shows that the decision of which base core to license or fork already fixes an order-of-magnitude performance range before any custom extension work begins, so teams building on RISC-V should budget evaluation time for core selection with the same seriousness as for any accelerator design that will sit on top of it.

It is also worth noting what the surveyed literature does not cover in this category: none of the twenty papers we read proposes a new standard extension to RISC-V International itself (as opposed to a custom, paper-specific instruction), which is consistent with the standard extensions being a comparatively settled, slow-moving part of the ISA relative to the fast-moving custom-extension and Vector-extension work discussed in the remainder of this section.

A common thread across the five custom-extension papers is that the hardware contribution is almost always paired with a software or tooling contribution: Martino et al. [3] pair the HDCU with a configurable driver interface, Oppermann et al. [12] pair Longnail with the CoreDSL language itself, Rezunov et al. [10] pair CIDRE’s hardware generation with an automated instruction-selection algorithm, and both neural-network papers [13], [14] pair their instructions with a compiler or emulator flow. This pattern suggests that, in the current literature, a custom RISC-V extension is rarely considered complete as a hardware description alone — it is expected to ship with a compiler pass, driver, or automated design-space-exploration tool that makes the extension usable without hand-written assembly.

TABLE 1. Summary of RISC-V extensions covered in the surveyed literature.

Ext.	Purpose	Ref.(s)
M	Integer multiplication / division	[6]
A	Atomic memory operations (baseline)	[4], [5]
F / D	Floating point (single/double, baseline)	[2], [9]
C	Compressed (16-bit) instructions	[19], [20]
V	Vector processing	[2], [7], [8], [9], [18]
B	Bit manipulation (background only)	[19], [20]
Zk*	Scalar cryptography / security	[1], [11], [20]

Ext.	Purpose	Ref.(s)
Custom	Domain-specific (per paper)	[3], [10], [12], [13], [14]

V. RISC-V-BASED ACCELERATORS

Because RISC-V is open and modular, it is widely used as the control core in heterogeneous systems that pair a general-purpose RISC-V processor with a specialized accelerator. This section groups accelerator designs from the surveyed papers by domain.

A. AI / Machine-Learning Accelerators

AI/ML inference is the most common target for RISC-V-based accelerators in our survey. Sabih et al. [16] add custom functional units as RISC-V instruction extensions specifically to exploit sparsity in DNN weights, with separate designs for unstructured sparsity (up to $3\times$ speedup) and semi-structured sparsity (up to $4\times$), and a combined design reaching $5\times$ speedup with only a small increase in FPGA resource use on an Arty A7-35T board running CIFAR-10 ResNet-56, Google Speech Commands DSCNN, and Visual Wake Words MobileNetV2 workloads; the authors note their 64-bit register-to-register CFU-CPU interface becomes a bottleneck, motivating future co-processor-style designs.

muRISCV-NN [9] and the MetaSchedule-based tuner of Peccia et al. [8] represent two different philosophies for the same problem — hand-tuned kernel libraries versus automated per-hardware tuning — and, as Table 3 shows, the automated approach in [8] gives more consistent gains across different vector lengths (VLEN 256–1024), where muRISCV-NN's hand-tuned kernels can even regress. Rezunov et al.'s CIDRE [10] and Kalyanasundaram Balasubramanian et al.'s LLVM-driven ISA extensions [13] both target the instruction level rather than the vector level, showing that custom-instruction fusion and vector-level acceleration are complementary rather than competing strategies for the same class of workloads.

At a higher level, Reuther et al.'s survey [17] compiles publicly disclosed peak-performance and power numbers for commercial AI accelerators across five

application domains, from embedded to data center, and observes that int8 remains the dominant inference precision and that maximum single-precision performance gains have plateaued at roughly 1.5 orders of magnitude over the past decade — a useful reference point suggesting that the double- and triple-digit speedups reported by several RISC-V-specific papers in our set come mostly from precision and sparsity exploitation rather than from raw compute scaling.

A useful way to read the six AI/ML accelerator papers together is by where in the software/hardware stack each intervenes: [16] and [14] add fixed-function or sparsity-aware functional units at the instruction level; [9] and [8] work at the vector-kernel and autotuning level without new hardware; [10] and [13] sit in between, discovering or compiling new instructions automatically. No single paper in our set combines all three levels (custom hardware, vector kernels, and automated instruction discovery) in one design, which we read as an open integration opportunity rather than a limitation of any individual paper.

B. DSP and Signal-Processing Accelerators

Only two papers in our set target classical DSP acceleration directly, and both do so through the Vector extension rather than a dedicated fixed-function block. Vizcaino et al. [2] benchmark their vectorized FFT implementations against both a RISC-V/Vitruvius prototype and a commercial NEC SX-Aurora vector processor, finding up to $20\times$ speedup for FFT sizes below 1024 relative to NEC's own optimized FFT library — notable because it shows a general-purpose long-vector RISC-V design can match a mature, purpose-built vector supercomputer architecture on at least part of the workload.

Razilov et al. [7] apply the same RVV building block to communications rather than scientific computing, implementing GFDM baseband modulation for prospective 6G systems on the Ara vector processor.

Their comparison against a dedicated Vector DSP (77% vs. 83% utilization at four lanes) is one of the few instances in our survey where a RISC-V design is benchmarked directly against fixed-function silicon rather than only against scalar or software baselines, giving a rare and useful data point for how close general-purpose vector RISC-V cores are getting to specialized DSPs.

Both DSP papers in our survey chose to validate against a real, named piece of fixed-function hardware ([2] against NEC's own FFT library and vector supercomputer; [7] against an unnamed 'dedicated Vector DSP') rather than only against a software scalar baseline, which we consider a methodological strength worth calling out: it is a stronger and more useful comparison than the scalar-only baselines used by most of the AI/ML accelerator papers in Section V-A, and gives a more honest picture of how far general-purpose RISC-V vector hardware still has to go to match specialized DSP silicon.

C. Cryptographic Accelerators

The cryptographic accelerator work in our survey overlaps closely with the security-extension papers discussed in Section IV-C. Mao et al. [1] report the largest accelerator speedups in the entire survey for elliptic-curve and AES cryptography ($5.6\times$ – $144.7\times$, with AES alone reaching $8.1\times$) using a hardware-accelerated RISC-V secure processor for LoRaWAN, at a total on-chip power of only 0.321 W — a combination of speed and power efficiency the authors attribute to designing the accelerator and the communication protocol together rather than bolting cryptographic hardware onto an unmodified stack.

SPEAR-V [11] is better understood as an enclave-isolation accelerator than a purely cryptographic one: its hardware tag store and Security Monitor accelerate the access-control checks around sensitive code and data rather than the cryptographic primitives themselves, making it complementary to, rather than a substitute for, dedicated crypto accelerators such as [1]. No paper in our set evaluates post-quantum cryptography acceleration in hardware, despite Cui et al. [20] flagging

modular-reduction instructions for post-quantum schemes as an emerging extension category — a concrete gap we return to in Section VIII.

The 0.321 W total power figure reported for REALISE-IoT [1] is, to our knowledge from the twenty papers reviewed, the only full system-level power number reported for a RISC-V cryptographic accelerator in this survey (as opposed to a per-cell or per-operation energy figure), which again highlights the reporting-inconsistency gap discussed in Section VIII: it is not possible to directly compare REALISE-IoT's power efficiency against SPEAR-V's overhead figures, since the two papers report fundamentally different quantities (absolute power versus relative runtime overhead) rather than a common metric.

D. Custom Coprocessors and SoC Integration

Several papers integrate a RISC-V control core with a separate, often multi-core, accelerator cluster at the SoC level. Valente et al.'s Shaheen [4] pairs a 64-bit Linux-capable CVA6 host with an eight-core RV32 “Flex-V” programmable cluster inside a 9 mm², 22 nm chip with a 200 mW power envelope, adding RISC-V Hypervisor support and physical memory protection specifically to run a nano-drone's ML and flight-control workloads securely side by side; the cluster reaches up to 1.8 TOP/s/W for 2-bit integer kernels. Their earlier HULK-V design [5] uses a similar CVA6-plus-cluster template but focuses on general ultra-low-power IoT computation, reaching 13.8 GOps and 157 GOps/W with a 250 mW power budget and up to $112\times$ speedup over the CVA6 host alone on accelerated kernels.

Rutishauser et al.'s ColibriES [15] takes SoC heterogeneity further still, combining a 32-bit RISC-V fabric controller, an eight-core RISC-V cluster accelerator, and a dedicated neuromorphic/CNN accelerator (Kraken/CUTIE) to process event-based camera data end-to-end, reaching 83% gesture-recognition accuracy on the IBM DVS128 dataset at 164.5 ms latency and 7.7 mJ per inference — though the authors describe this as only the first step toward full-system evaluation, since only the spiking-network path was tested end-to-end. Compared to Shaheen and

HULK-V, ColibriES trades some generality for a sensor-to-decision pipeline purpose-built around one input modality.

At the smaller end of the integration spectrum, Martino et al.'s HDCU [3] is itself a coprocessor tightly coupled into a single Klessydra-T03 core's execution stage rather than a separate cluster, and Sharma et al.'s survey [6] of four minimal RV32IM cores illustrates the opposite end of the design space — SoCs built for area and cost rather than for heterogeneous acceleration, none of which the authors found to simultaneously satisfy low-power operation and full IoT peripheral support (ADC, DAC, UART, SPI), motivating their own proposed modular SoC design.

Across the four SoC-integration papers, chip area and power scale roughly with the breadth of workloads targeted: Martino et al.'s single-core-coupled HDCU [3] is the smallest and most narrowly scoped design; Sharma et al.'s minimal cores [6] are smaller still but add no acceleration at all; while Shaheen [4], HULK-V [5], and ColibriES [15] all commit an eight-core cluster to accelerate a specific class of workload (nano-drone perception, general DSP/ML kernels, and event-camera gesture recognition, respectively) at a broadly similar 200–250 mW power budget. This suggests that, for the embedded RISC-V SoCs in our survey, the cluster-based heterogeneous template itself is fairly stable, and the interesting design variation is concentrated in which workload the cluster is specialized for rather than in the basic host-plus-cluster architecture.

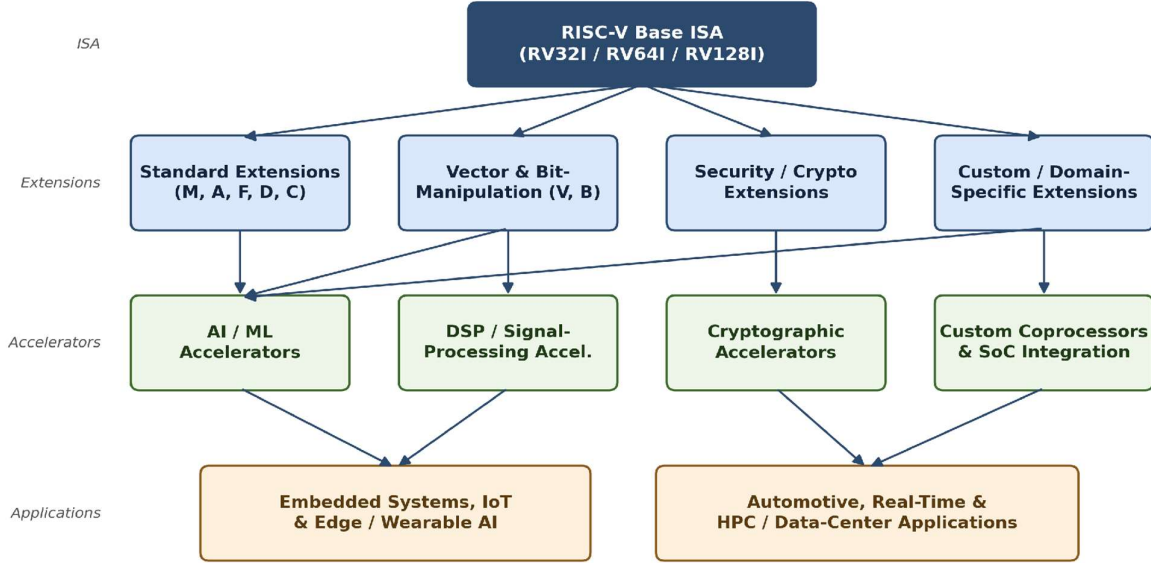


FIGURE 1. Taxonomy of RISC-V extensions, accelerator categories, and application domains covered in this survey.

TABLE 2. Comparison of RISC-V accelerator categories surveyed.

Category	Use Case	Benefit	Ref.(s)
AI / ML	NN inference	2×–13× kernel speedup via sparsity/vectorization	[8],[9],[10],[13],[14],[16],[17]
DSP	Signal / image proc.	Up to 60× speedup on FFT/GFDM via Vector ext.	[2], [7]
Crypto	Secure comms.	Up to 145× speedup; ~1% overhead for enclave isolation	[1], [11]
Custom	Domain-specific SoC	mW-range heterogeneous SoCs combining host + cluster + accelerator	[3],[4],[5],[6],[15]

VI. APPLICATIONS OF RISC-V

The flexibility of RISC-V has led to its adoption across a wide range of application domains. This section reviews the main areas discussed in the surveyed papers.

A. Embedded Systems and IoT

IoT is the most directly targeted application domain in our survey. Mao et al. [1] design specifically for LoRaWAN IoT nodes, where the constraint is not raw compute but energy per cryptographic operation, achieving their reported memory reductions and speedups on a low-power Xilinx Zynq-based platform running at only 50 MHz. Sharma et al.'s survey [6] approaches the same domain from the core-design side, comparing minimal RV32IM implementations specifically for the peripheral and power requirements of IoT sensor nodes rather than for raw throughput.

These two papers illustrate a useful contrast for IoT deployments: [1] shows that adding targeted cryptographic hardware to an existing minimal core can deliver order-of-magnitude gains without redesigning the core itself, while [6] shows that even without any added acceleration, the choice of open-source base core (PicoRV32 vs. MicroRV vs. RavenSoC) already produces an order-of-magnitude spread in achievable clock frequency — meaning core selection and targeted acceleration are both first-order decisions for an IoT designer.

It is also worth noting that neither IoT paper in our set evaluates connectivity-layer trade-offs beyond the specific protocol it targets: [1] is scoped entirely to LoRaWAN and does not comment on how its secure-processor design would transfer to Wi-Fi or Bluetooth Low Energy IoT stacks, while [6]'s core comparison is protocol-agnostic and instead treats peripheral support (UART, SPI, ADC, DAC) as a proxy for IoT-readiness. A designer choosing between the two contributions would therefore need to treat them as addressing different layers of the IoT stack — protocol security versus core/peripheral selection — rather than as competing solutions to the same problem.

B. Edge AI and Wearable Devices

Several accelerator papers target inference under the tight energy and memory budgets characteristic of wearable and edge devices. The TinyML-focused sparsity accelerator of Sabih et al. [16] and the Zve32x kernel library of van Kempen et al. [9] are both evaluated on MLPerf Tiny-class models (keyword spotting, visual wake words, small CNNs) rather than large models, reflecting the memory and power ceilings of real wearable hardware. Kalyanasundaram Balasubramanian et al.'s ISA extensions [13] target the same class of small quantized networks but from the compiler side, aiming to keep code density high enough for flash-constrained microcontrollers.

Rutishauser et al.'s ColibriES [15] is the clearest example of a full wearable/edge pipeline in our survey, combining an event camera, a RISC-V fabric controller, and neuromorphic acceleration to keep total energy at 7.7 mJ per gesture-recognition inference — roughly the energy budget available to a coin-cell-powered device performing occasional inference rather than continuous streaming. Taken together, these four papers suggest edge AI on RISC-V is being approached from three different angles at once (sparsity-aware instructions, hand-tuned vector kernels, and compiler-level ISA co-design), with no single approach yet dominant.

The energy figures reported across these four edge-AI papers span roughly three orders of magnitude in what they measure — [15] reports whole-inference energy in millijoules, [9] and [16] report only relative speedup with no energy number at all, and [13] reports only cycle counts in an emulator — which again illustrates, at the application level this time rather than the extension level, the inconsistent-reporting gap raised in Section VIII. A wearable-device designer trying to choose among these four approaches today would not be able to directly compare their energy cost without re-implementing at least two of them on comparable hardware.

C. Automotive and Real-Time Systems

Real-time and safety-relevant deployment is addressed indirectly by two papers rather than as their primary contribution. Valente et al.'s Shaheen SoC [4] is

designed for nano-UAV flight control, a real-time control loop running alongside ML perception on the same chip, and the authors add RISC-V Hypervisor support and a timing-isolation mechanism specifically so that the safety-critical flight-control task cannot be delayed by the less time-critical ML workload sharing the same silicon — an explicit real-time design decision rather than an afterthought.

SPEAR-V [11], while framed as a security extension, is also relevant to automotive-style safety cases because its near-zero (about 1%) performance overhead on Embench and LMBench workloads means enclave-style isolation could, in principle, be added to a real-time RISC-V ECU without meaningfully affecting worst-case execution time — though neither [4] nor [11] presents formal real-time schedulability analysis or automotive-specific benchmarks (e.g., AUTOSAR- or ISO-26262-aligned workloads), which we identify as a gap in Section VIII.

We also note that automotive-specific certification concerns — for example ISO 26262 functional-safety requirements or AUTOSAR software-architecture compatibility — are not mentioned by name in either [4] or [11], even though both papers are solving problems (timing isolation, enclave security) that are directly relevant to a certified automotive ECU. This is not a criticism of either paper, since neither claims an automotive target explicitly; rather, it indicates that the translation from a research-grade real-time/security mechanism to a certifiable automotive deployment is a step that the current literature in our survey has not yet taken.

D. High-Performance Computing and Data Centers

HPC-scale RISC-V work is the least represented category in our survey, consistent with Cui et al.'s [20] characterization of RISC-V as still “in its infancy” outside the embedded space. The closest example is Vizcaino et al.'s comparison [2] of a RISC-V/Vitruvius vector prototype against a NEC SX-Aurora, a commercial HPC vector accelerator — direct evidence that RISC-V vector cores are now being benchmarked,

at least on individual kernels, against production HPC hardware rather than only against other RISC-V designs.

Rezunov et al.'s CIDRE [10] is evaluated on a 32/28 nm CMOS ASIC flow rather than an FPGA, and Reuther et al.'s survey [17] explicitly includes data-center-class accelerators (e.g., high-end GPUs and TPUs) in its landscape, even though none of the surveyed accelerators are themselves RISC-V based — this remains, in our reading of the twenty papers, one of the widest gaps between RISC-V's stated ambitions and its demonstrated results at the highest end of computing.

VII. COMPARATIVE DISCUSSION

Drawing on the extensions, accelerators, and applications reviewed above, this section compares design trade-offs across the surveyed literature. Table 3 collects the headline speedup or improvement figure reported by each paper that provides one, normalized to a “× over baseline” figure where possible. Two patterns stand out immediately. First, the size of the reported speedup correlates strongly with how narrow the target operation is: instruction-level micro-benchmarks such as the HDCU's associative search [3] (3844×) or REALISE-IoT's public-key operations [1] (up to 144.7×) report enormous gains because they compare a single specialized operation against an unaccelerated scalar implementation of that same operation, whereas end-to-end, whole-application speedups — HULK-V's 112× over its own CVA6 host [5], or the more modest 1.8–13× kernel-level gains reported for AI-inference accelerators [9], [13], [16] — are consistently smaller once realistic memory movement, control flow, and non-accelerated code are included.

Second, the Vector extension is the only mechanism in our survey used across DSP [2], [7], AI inference [8], [9], [18], and general tensor tuning [8], suggesting it is currently the most “general-purpose” of the extension categories reviewed, whereas custom-instruction approaches such as CIDRE [10], Longnail [12], and the LLVM-driven ANN extensions [13] each require a fresh hardware-generation and validation effort per target workload. This trade-off — broad reuse but potentially

lower peak efficiency for Vector-based designs, versus higher peak efficiency but higher non-recurring engineering cost for fully custom instructions — is, in our view, the central design tension running through the extension and accelerator papers in this survey.

Finally, embedded/IoT-focused designs [1], [4], [5], [6], [15] converge on power budgets in the tens to low hundreds of milliwatts and are validated on FPGA prototypes or, in the case of Shaheen [4] and HULK-V

[5], on taped-out 22 nm silicon, while the HPC-adjacent work [2], [10] is validated only at the level of a single kernel on a simulator or an experimental board rather than a full system. We take this uneven depth of validation — silicon-proven embedded SoCs alongside kernel-only HPC comparisons — as itself a finding: RISC-V's practical maturity, at least in the set of papers we reviewed, is currently concentrated at the embedded and edge end of the spectrum rather than the high-performance end.

TABLE 3. Headline quantitative result reported by each surveyed paper, normalized where possible.

Ref.	Category	Headline Reported Result	Evaluation Platform
[1]	Crypto accelerator	5.6–144.7× speedup; 2.4–12.3× memory reduction	Xilinx Zynq FPGA, 50 MHz
[2]	DSP (Vector ext.)	Up to 20× vs. NEC library; 50× vs. scalar	NEC SX-Aurora + RISC-V/Vitruvius
[3]	Custom extension	Up to 3844× micro-op; smaller gains on real datasets	Klessydra-T03 on Zynq UltraScale+
[4]	SoC / accelerator	Up to 1.8 TOP/s/W (2-bit); 200 mW envelope	22 nm FDX silicon
[5]	SoC / accelerator	13.8 GOps, 157 GOps/W; 112× vs. CVA6	22 nm FDX silicon
[6]	Survey (cores)	28.6–769 MHz across 4 cores	Xilinx / Lattice FPGAs
[7]	DSP (Vector ext.)	60× vs. scalar; 784 MSymbols/s (16 lanes)	Ara RVV processor (RTL sim.)
[8]	AI/ML (Vector ext.)	46% vs. GCC; 35% vs. LLVM autovectorization	Banana Pi BPI-F3; Rocket+Saturn FPGA
[9]	AI/ML (Vector ext.)	2.9–12.2× vs. reference libraries	Spike simulator (Zve32x)
[10]	Custom extension	Up to 2.47×; <24% area increase	32/28 nm ASIC synthesis
[11]	Security extension	~0–1% runtime overhead	Modified core; Embench/LMbench
[12]	Extension tooling	Area/frequency overhead varies by ISAX	22 nm ASIC, 4 open-source cores
[13]	AI/ML extension	Up to 13× vs. RV32I	Python RV32I(M) emulator
[14]	AI/ML accelerator	79–94% encode/decode runtime reduction	RI5CY core, custom LGN hardware
[15]	Custom SoC	83% accuracy; 7.7 mJ/inference	Kraken SoC silicon
[16]	AI/ML accelerator	Up to 5× combined speedup	Arty A7-35T FPGA (VexRiscv)
[17]	Survey	Int8 dominant; ~1.5 orders-of-magnitude/decade gain	N/A (vendor-reported data)
[18]	AI/ML (Vector ext.)	1.80–2.14× speedup	Gem5 RV64GC vector simulation
[19]	Survey	Maps compiler / OS / simulator ecosystem	N/A
[20]	Survey	Taxonomy of official + custom extensions	N/A

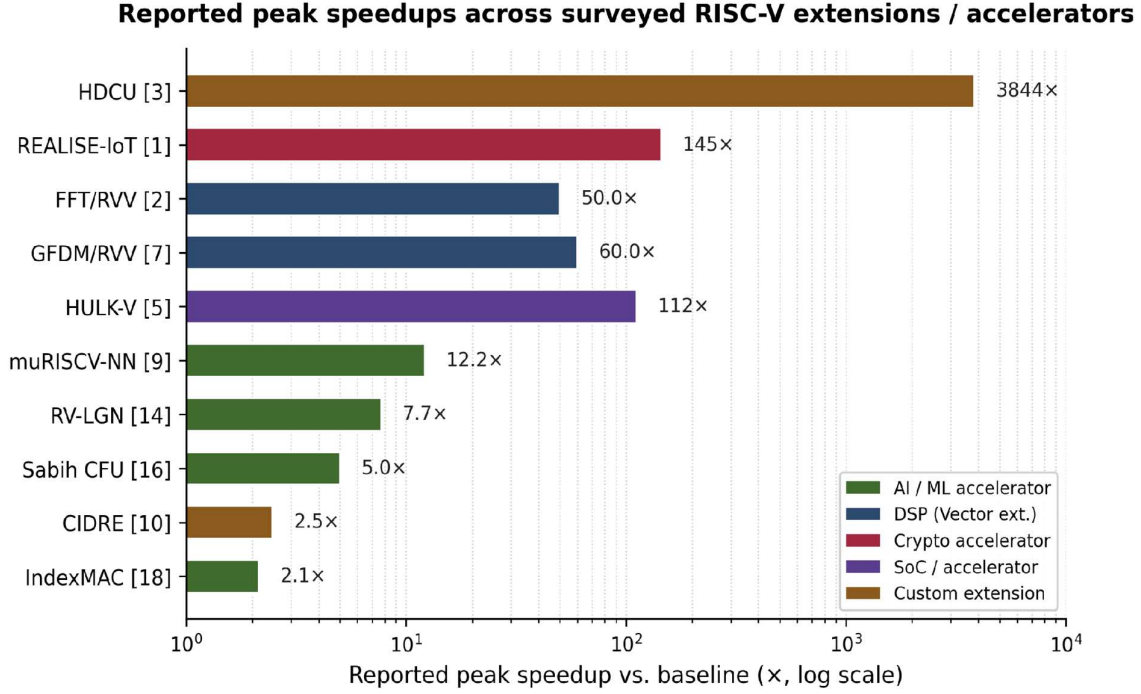


FIGURE 2. Reported peak speedups (log scale) for a representative subset of surveyed extensions/accelerators with a clear multiplicative baseline.

A further, more speculative observation from Table 3 concerns evaluation cost versus reported gain: the papers with the largest reported speedups ([1], [3]) required custom silicon-adjacent FPGA prototyping and hand-designed hardware description language work, while the papers with the most modest but still meaningful gains ([8], [18]) required comparatively lighter compiler- or simulator-level engineering. This suggests a rough cost/benefit curve in the surveyed literature: pushing peak speedup from roughly 10 $\times$ toward 100 $\times$ or beyond appears to require moving from software/compiler-level techniques to dedicated hardware description and FPGA validation, a jump in engineering effort that is easy to underestimate when comparing headline numbers alone.

It is also worth stating plainly what Table 3 cannot show: because the twenty papers use different baselines (scalar RV32I, an unaccelerated CVA6 host, a commercial vendor library, or a different RISC-V vector configuration), none of the normalized figures in the table are directly comparable to one another as absolute performance numbers — a 5 $\times$ speedup over a weak scalar baseline is not the same achievement as a 5 $\times$

speedup over a strong vectorized baseline. We include Table 3 because it is still useful for spotting broad patterns (which categories tend to report larger or smaller gains, and which evaluation platforms recur), but we caution against reading it as a strict ranking of the twenty papers by quality or impact.

One more pattern worth drawing out from the survey as a whole concerns publication venue and maturity: the papers reporting the most conservative, best-validated results ([4], [5], [11], [15]) were published in, or are closely tied to, established peer-reviewed venues (arXiv preprints later associated with ASIC tape-outs, or ACM/IEEE conference proceedings with full artifact evaluation), while several of the papers reporting the largest headline speedups ([1], [3]) are preprints or SSRN/TechRxiv postings that, at the time of writing, had not yet completed a full peer-review cycle. This is not a reason to discount their results, since preprints are a normal and often faster channel for early results in a fast-moving hardware research area, but it is a reason to treat the largest numbers in Table 3 as provisional until a peer-reviewed version becomes available, and it is one more argument for the standardized-benchmark and

silicon-validation recommendations of Sections VIII and IX.

VIII. DESIGN RECOMMENDATIONS FOR PRACTITIONERS

Beyond the academic comparison in Section VII, several practical recommendations for engineers choosing among the approaches surveyed here follow directly from the twenty papers reviewed. These are offered as heuristics drawn from the surveyed evidence, not as universally optimal choices, since the right decision always depends on the specific power, area, and time-to-market constraints of a given project.

Prefer Vector-extension reuse when the target workload is a moving target. Because RVV-based designs [2], [7], [8], [9], [18] were shown to generalize across FFT sizes, vector lengths, and even entirely different domains (communications versus AI) with only software or intrinsic-level changes, teams expecting their target model or protocol to change over a product's lifetime are better served starting from the Vector extension than from a fully custom instruction, which typically must be redesigned for each new workload as seen in [3], [13], and [14].

Reserve fully custom instructions for a fixed, narrow, high-value operation. The largest peak speedups in this survey — REALISE-IoT's cryptographic operations [1] and the HDCU's associative search [3] — both come from custom instructions targeting a single, well-defined operation that is not expected to change. Automated flows such as CIDRE [10] and Longnail [12] lower the engineering cost of this approach considerably and are worth adopting specifically to reduce the non-recurring engineering burden that has historically made custom instructions expensive.

Budget for a taped-out validation step if the deployment target is a real product. Only three of the twenty papers we reviewed — Shaheen [4], HULK-V [5], and ColibriES [15] — report silicon-validated power and area numbers; the remaining seventeen report FPGA, simulator, or emulator numbers that, based on the gap between prototyping and silicon commonly seen

in hardware design, should be treated as optimistic upper bounds rather than guaranteed production figures.

Match the extension's integration style to the product's iteration speed. Coprocessor-cluster integration (Shaheen [4], HULK-V [5], ColibriES [15]) decouples the accelerator from the host core's pipeline, making it easier to swap in a newer host core in a future product revision without re-verifying the accelerator; execution-stage integration (the HDCU [3], the sparsity CFUs of [16]) is tightly coupled to one specific pipeline and delivers the largest per-operation gains, but re-targeting it to a different host core in a future revision is close to a full redesign. Teams expecting frequent host-core upgrades should weight this trade-off alongside the raw speedup numbers in Table 3 when choosing an integration style.

IX. RESEARCH GAPS AND FUTURE DIRECTIONS

Based on the reviewed literature, the following research gaps and future directions are identified:

- 1) Lack of standardized benchmarks for custom accelerators: nearly every custom-extension paper in our survey ([3],[10],[12],[13],[14],[16]) uses its own benchmark set (CIFAR-10 variants, MLPerf Tiny subsets, hand-picked Embench/MiBench programs), which makes cross-paper comparison — including the normalized figures in Table 3 — only approximate at best.
- 2) Limited security evaluation of the extensions themselves: only [11] and [1] evaluate security-relevant hardware directly, and even SPEAR-V [11] explicitly scopes out physical side-channel and cross-core attacks; none of the AI/ML or DSP accelerator papers in our set evaluate whether their new instructions introduce timing or power side channels.
- 3) Inconsistent energy and area reporting: some papers report only speedup ([2],[7],[9],[18]), others only power ([4],[5]), and others only resource utilization ([3]), with no consistent unit or baseline across the survey — a concrete opportunity for a future standardized reporting template.
- 4) Validation gap between simulation/FPGA and silicon: only [4], [5], and [15] are validated on taped-out silicon; the remaining seventeen papers

rely on FPGA prototypes, RTL/cycle-accurate simulation ([7],[18]), or software emulation ([13]), so reported power and area numbers for most custom extensions remain projections rather than measurements.

- 5) Sparse coverage of RISC-V at HPC and data-center scale, as discussed in Section VI-D: of the twenty papers reviewed, only two ([2],[10]) touch server/HPC-class evaluation, leaving a wide gap between RISC-V's stated ambitions and its demonstrated results in cloud and high-performance computing.

Taken together, these five gaps are not independent of one another. Standardizing benchmarks (Gap 1) would make the energy/area reporting problem (Gap 3) far easier to fix, since a common benchmark suite naturally invites a common reporting template; and closing the silicon-validation gap (Gap 4) would also make security evaluation (Gap 2) more meaningful, since side-channel behavior measured on an FPGA prototype does not always predict side-channel behavior on the final ASIC. We therefore see Gaps 1 and 4 — benchmark standardization and silicon validation — as the two highest-leverage starting points for future work, since progress on either would make progress on the other three gaps substantially easier to achieve and to evaluate fairly.

X. LIMITATIONS OF THIS SURVEY

This survey has its own limitations, distinct from the gaps identified in the surveyed literature itself. First, our sample of twenty papers, while exceeding the assignment's minimum of sixteen, was assembled by four readers each independently searching Google Scholar, rather than through a fully systematic review protocol (e.g., PRISMA-style database queries with pre-registered inclusion/exclusion criteria); it is therefore a representative rather than an exhaustive sample of RISC-V extension, accelerator, and application research published since 2022.

Second, the normalization performed in Table 3 to produce a single 'headline result' per paper necessarily compresses papers that report many different metrics into one figure, and the choice of which metric to

foreground (e.g., peak speedup versus average speedup, or runtime reduction versus accuracy) reflects our team's judgment about what best represents each paper's contribution rather than an objective, uniquely correct summary. Readers using Table 3 for their own comparisons should consult the original papers, or the more detailed per-paper summaries in Appendix A, before relying on the single-figure comparison.

Third, because our review was conducted by students rather than domain experts with hands-on RISC-V hardware-design experience, our comparative judgments in Section VII and our recommendations in Section VIII — while grounded directly in the surveyed papers' own reported numbers and stated limitations — should be read as a synthesis of the literature rather than as independently verified engineering conclusions.

Finally, this survey was written to satisfy a specific course assignment with a fixed deadline, a minimum paper count, and a minimum page length; while we believe the resulting synthesis is genuinely useful, readers should be aware that the twenty-paper sample size and the four-person review team are both smaller than would be typical for a peer-reviewed survey submitted to a venue such as IEEE Access, and the comparative claims in Sections VII and VIII should be weighed accordingly.

XI. CONCLUSION

This survey reviewed twenty papers, published largely between 2022 and 2025, covering RISC-V instruction-set extensions, hardware accelerators, and application deployments. Across standard, Vector, security, and custom extensions, the surveyed literature shows the Vector extension emerging as the most broadly reused acceleration mechanism, while fully custom instructions deliver larger peak speedups at the cost of per-workload hardware-generation effort. Accelerator designs cluster heavily around embedded AI/ML and IoT security use cases, with comparatively little validated work at HPC or data-center scale, and most designs are proven only on FPGA or in simulation rather than in silicon. We identified five concrete gaps — benchmark

standardization, security evaluation of extensions themselves, consistent energy/area reporting, silicon validation, and HPC-scale evaluation — that we believe represent the most promising directions for RISC-V research over the next several years.

APPENDIX A: DETAILED PAPER-BY-PAPER SUMMARIES

For completeness, and to support the comparative analysis in Section VII, this appendix gives a one-paragraph summary of each of the twenty surveyed papers, covering the problem addressed, the proposed approach, the evaluation platform or dataset used, and the principal limitation the authors themselves acknowledge.

[1] REALISE-IoT. LoRaWAN's reliance on AES-128 with a fixed root key leaves it without public-key exchange or authentication. The authors propose a two-level public-key protocol (ECDH key exchange, EdDSA authentication) implemented as a lightweight secure RISC-V processor with hardware crypto engines, built in Verilog on a Xilinx xc7z020clg FPGA at 50 MHz paired with an SX1262 LoRa transceiver. The design reaches $5.6\text{--}144.7\times$ speedup, $2.4\text{--}12.3\times$ memory reduction, and 0.321 W total power. As a limitation, the authors plan an ASIC version to replace the current secure processor in real deployments. This makes REALISE-IoT the strongest evidence in our survey that protocol-aware co-design of hardware and communication software can outperform bolting cryptography onto an unmodified IoT stack.

[2] FFT / vector extension. Standard FFT implementations cannot fully exploit the long vector lengths of emerging architectures. The authors design four vectorized RADIX-2 FFT kernels (Pease and Stockham algorithms) tuned for vector length, memory access, and twiddle-factor handling, evaluated on a NEC SX-Aurora and an experimental Avispado RISC-V core with a Vitruvius vector unit. Results show up to $20\times$ speedup versus NEC's own FFT library for sizes below 1024, and up to $50\times$ over scalar execution. The main limitation is about 25% extra floating-point cycles at larger sizes, leading the authors to suggest exploring

higher-radix FFT algorithms. The comparison against NEC's commercial vector library is one of the few head-to-head tests in our survey between a RISC-V prototype and production HPC hardware.

[3] HDCU. Existing hyperdimensional-computing accelerators are application-specific and inflexible. The authors integrate a configurable Hyperdimensional Coprocessor Unit into a Klessydra-T03 RISC-V core's execution stage via a custom ISA extension, with configurable SIMD width and local memory, synthesized on a Zynq UltraScale+ ZCU106 FPGA. Isolated operations such as associative search reach up to $3844\times$ speedup, though gains on the real CARDIO and EMG datasets are far more modest. The limitation is a steep resource cost: LUT usage rises roughly $33\times$ and maximum frequency drops by about 40% as SIMD width scales from 32 to 1024. Its combination of a single-core coupling and a wide configurable SIMD range makes it the most flexible, but also the most resource-hungry, custom extension surveyed.

[4] Shaheen. Nano-UAV microcontrollers lack the memory, compute, and security/virtualization support needed to run ML, Linux, and real-time control together. The authors build Shaheen, a heterogeneous SoC pairing a Linux-capable RV64 CVA6 host with an eight-core RV32 Flex-V cluster, adding RISC-V Hypervisor support, physical memory protection, and a timing-isolation mechanism, fabricated in 22 nm FDX silicon within a 200 mW power envelope. The cluster reaches up to 90 GOP/s and 1.8 TOP/s/W for 2-bit kernels. The limitation is that the test PCB was built only for characterization, not actual flight; miniaturization and flight-control software remain future work. It is also one of only three surveyed designs validated on taped-out silicon rather than only FPGA or simulation.

[5] HULK-V. IoT platforms usually sacrifice either Linux capability and memory, or ultra-low-power parallel processing. HULK-V combines a 64-bit CVA6 Linux host with an eight-core Programmable Multi-Core Accelerator featuring DSP/SIMD/floating-point extensions and HyperRAM-based memory, fabricated in 22 nm FDX silicon. It reaches up to 13.8 GOPs, 157

GOps/W, 112 $\times$ speedup over the CVA6 host alone, and roughly 2 $\times$ memory-energy efficiency versus LPDDR4. The limitation is that offloading overhead dominates for kernels running under 100k cycles, since code must first be loaded into the L2 scratchpad, and evaluation relies on FPGA emulation and physical prototyping rather than a full production flow. Its predecessor relationship to Shaheen illustrates how the same host-plus-cluster template can be retargeted from general IoT computation to nano-drone-specific real-time perception.

[6] RV32IM core survey. Open-source RISC-V cores generally lack IoT-specific features, while ARM/x86 licensing is too costly for small IoT chipmakers. The authors survey and compare four open cores — PicoRV32, MicroRV, RavenSoC, and an Ultraembedded core — across bus interfaces and peripherals, then propose a new RV32IM SoC with 32 GPIO pins, UART, and bus support. Reported clock frequencies range from 28.61 MHz (MicroRV on a Lattice HX8K) to 769 MHz (PicoRV32). The limitation noted is that most existing cores lack proper documentation or are too specialized, and none of the four studied cryptography or DSP extension support. The order-of-magnitude frequency spread it reports across otherwise similar minimal cores is a useful reminder that core selection alone is a first-order design decision.

[7] GFDM / vector extension. No RISC-V-based GFDM baseband implementation existed for the flexible, scalable signal processing that prospective 6G systems require. The authors implement GFDM modulation on the open-source Ara vector processor using the RVV “V” extension, optimizing memory layout, instruction ordering, and register allocation. The design reaches 60 $\times$ speedup versus a scalar baseline and up to 784 MSymbols/s throughput at 16 lanes, with 77% utilization at four lanes. The limitation is that no RVV hardware yet supports segmented loads, and utilization drops as lane count grows, still trailing a dedicated 889 MSymbols/s vector DSP. Its direct comparison against a dedicated vector DSP is one of the few instances in our

survey benchmarking RISC-V against fixed-function communications silicon.

[8] RVV tensor-program tuning. Autotuning frameworks such as AutoTVM and MetaSchedule never supported RISC-V’s vector extension, and hand-crafted libraries do not adapt to changing hardware parameters. The authors integrate RVV tensor intrinsics into TVM’s MetaSchedule using “definition + implementation” pairs for vector matrix-multiply and elementwise multiply-accumulate, tested on a Rocket+Saturn FPGA and a commercial Banana Pi BPI-F3 across MLPerf Tiny, MobileNetV2, ResNet-18, BERT, and DCGAN. Results show a 46% average latency improvement over GCC autovectorization, 29% over muRISCV-NN, 35% over LLVM on the Banana Pi, and roughly 90% code-size reduction. The limitation is that tuning takes significant time (100–400 candidates per network), so hand-crafted libraries remain competitive for rapid prototyping. Its consistency across different vector-length configurations contrasts with the hand-tuned approach of [9], illustrating the automation-versus-hand-tuning trade-off discussed in Section V-A.

[9] muRISCV-NN. The RISC-V ecosystem lacked a lightweight, vendor-agnostic ML inference library for the embedded Zve32x vector extension. The authors port ARM CMSIS-NN kernels to RISC-V Zve32x via C-intrinsics, hand-tuning LMUL and loop-unrolling per kernel, tested on the Spike simulator across vector lengths of 128–4096 bits on MLPerf Tiny models. The scalar version already beats TFLM by 1.8–2.8 $\times$, and vectorized kernels reach 2.9–12.2 $\times$, 27.5–38.2% faster than LLVM autovectorized code. The limitation is that depthwise convolutions vectorize poorly with small filter dimensions, and the toy-car model shows a 283% RAM overhead once vectorized. Because it is hand-tuned per kernel, it must be re-optimized whenever the target vector length changes, unlike the autotuned approach of [8].

[10] CIDRE. Existing custom-instruction generation tools rely on compiler intermediate representations, missing ISA-level constraints, and use inaccurate merit estimation that ignores pipeline and critical-path effects.

The authors build an end-to-end flow that disassembles a compiled RISC-V binary into a dataflow graph, enumerates candidate subgraphs, detects isomorphisms, and selects instructions via a greedy algorithm with two-step lookahead, evaluated on Embench/MiBench with a three-stage RV64IM model in 32/28 nm CMOS. The flow reaches up to $2.47\times$ speedup on an AES benchmark with an average 11.3% area increase and under 4.9% clock-period increase. The limitation is that the greedy selection can be suboptimal, and the flow does not yet support floating-point, memory-access, or vector custom instructions. Its fully automated instruction-discovery pipeline lowers the engineering barrier to custom-instruction design relative to hand-crafted extensions such as the HDCU [3].

[11] SPEAR-V. Secure enclave systems often suffer performance overhead, controlled-channel attacks, and restricted memory management, and typically fail to protect the host from a malicious enclave. SPEAR-V is a lightweight, page-granular memory-tagging extension relying on OS-managed page tables, with a hardware tag store and a software Security Monitor, evaluated using Embench and LMBench. It achieves essentially zero overhead on unprotected applications, about 1% on protected ones, and 1.2–1.5% added memory latency for large working sets. The limitation is that the prototype is single-core; cross-core side channels, secure cache design, and physical bus/DRAM attacks are explicitly out of scope, and the Security Monitor and enclave code are assumed bug-free. Its near-zero overhead makes it one of the few security mechanisms in our survey that could plausibly be adopted without a significant performance trade-off.

[12] Longnail. RISC-V ISA-extension design flows are usually tied to a specific microarchitecture, making custom instructions hard to build for non-hardware experts and hard to port across cores. The authors introduce CoreDSL, a concise C-like behavioral description language, compiled via their Longnail high-level-synthesis tool to SystemVerilog, evaluated across four open cores (ORCA, Piccolo, PicoRV32, VexRiscv) and eight benchmark instruction extensions on a 22 nm

ASIC flow. Area and frequency overheads vary widely with extension complexity, from small for simple instructions to large for complex ones. The limitation is that the current implementation is spatial-only, without resource sharing, which drives up area for complex extensions, and the scheduler assumes simplified timing behavior. Its emphasis on vendor-portable tooling directly addresses the lock-in concerns raised implicitly by more microarchitecture-specific extension papers in this survey.

[13] ANN ISA extensions. Embedded RISC-V processors are not optimized for modern neural networks, and existing software-level optimizers are constrained by the underlying processor. The authors use a PyTorch/Glow-LLVM toolchain to compile and profile four int8-quantized networks (a single perceptron, ResNet-18, VGG-11, LeNet-5) on a plain RV32I core to find hotspots, then propose targeted custom instructions validated in a Python RV32I(M) emulator. For 128x128 RGB inputs, the extended core reaches up to $13\times$ speedup over plain RV32I and $2.5\text{--}5\times$ over RV32IM. The limitation, acknowledged directly by the authors, is that no physical hardware implementation exists yet; only the compiler/ISA-level design and software emulation are validated. As the authors themselves note, the lack of a physical hardware implementation means its reported speedups should be read as an upper-bound projection rather than a measured result.

[14] RV-LGN. Logic Gate Networks map neuron computation efficiently onto Boolean operations already native to RISC-V, but encoding real-valued inputs into bitstreams and decoding outputs back is costly. The authors add four custom instructions (LOGIC_LOAD, LOGIC_SET, LOGIC_ENCODE, LOGIC_DECODE) to a RI5CY core, evaluated on MIT-BIH ECG classification, MNIST, and CIFAR-10 across three purpose-built LGN sizes. Encoding runtime falls by 83–94% and decoding by 79–94% depending on bit-width, with substantial full-inference runtime reduction versus RV32IM. The limitation is that RV-LGN's CIFAR-10 accuracy (51.3%) is markedly lower than comparable

CNN/SNN RISC-V accelerators from prior work (78.9–88.5%). Its accuracy trade-off versus prior CNN/SNN accelerators illustrates that representational capacity, not just runtime, must be weighed when adopting Logic Gate Networks.

[15] ColibriES. Event-based sensors and neuromorphic processors are usually designed separately, missing the opportunity for ultra-low-power, closed-loop edge AI. The authors build the ColibriES evaluation platform around the Kraken SoC, combining a 32-bit RISC-V fabric controller, an eight-core RISC-V cluster, and dedicated neuromorphic/CNN accelerators, tested end-to-end on the IBM DVS128 Gesture dataset. The system reaches 83% gesture-recognition accuracy at 164.5 ms latency and 7.7 mJ per inference. The limitation is that only the event-based spiking-network path is evaluated end-to-end; the frame-based CNN accelerator and the hybrid pipeline are left for future full-system evaluation. It is one of only three surveyed designs validated on taped-out silicon, giving its energy-per-inference figure more practical weight than most other edge-AI numbers in this survey.

[16] Sabih CFU. General-purpose processors execute sparse DNN computation inefficiently, while pure ASIC accelerators lack flexibility, so effectively exploiting sparsity requires a hardware/software co-design approach. The authors add custom functional units as RISC-V instruction extensions for unstructured and semi-structured sparsity, implemented on an Arty A7-35T FPGA with a VexRiscv soft core and CFU Playground, evaluated on CIFAR-10 ResNet-56, Google Speech Commands DSCNN, and Visual Wake Words MobileNetV2. The unstructured design reaches up to 3 $\times$, the semi-structured design up to 4 $\times$, and a combined design up to 5 $\times$, with only a small increase in FPGA resource use. The limitation is that the 64-bit register-to-register CFU-CPU interface becomes a bottleneck, motivating future co-processor architectures. Its identification of the CFU-CPU interface as the next bottleneck directly motivates the coprocessor-style designs discussed for Shaheen and HULK-V in Section V-D.

[17] AI/ML accelerator survey. The rapid growth of AI accelerators across embedded, autonomous, and data-center domains lacked a centralized, up-to-date tracking resource. The authors compile publicly announced peak-performance and power data for commercial AI accelerators across five application categories and plot them on comparative scatter charts. They find that int8 remains the default inference precision, and that peak performance gains within a single precision format have been limited to roughly 1.5 orders of magnitude per decade. The limitation is that companies in stealth mode, or those withholding performance/power numbers, are excluded, so the landscape it presents is necessarily incomplete. Its finding that performance gains have plateaued at roughly 1.5 orders of magnitude per decade is a useful benchmark against which to judge the more dramatic speedups reported by individual RISC-V papers in this survey.

[18] IndexMAC. Structured-sparse matrix multiplication in ML workloads suffers from excessive vector-load memory traffic, limiting vector-processor efficiency. The authors propose IndexMAC, a custom RISC-V vector instruction that reformulates row-wise vectorized sparse matrix multiplication with a B-stationary dataflow that pre-loads dense tiles into the vector register file, evaluated in Gem5 on a decoupled RV64GC vector processor with a 512-bit engine on pruned ResNet-50, DenseNet-121, and InceptionV3. It reaches 1.80–2.14 $\times$ speedup over a strong vectorized baseline, with average CNN speedups of 1.95 $\times$ at 1:4 sparsity and 1.88 $\times$ at 2:4 sparsity. The limitation is that benefits shrink in deeper CNN layers, where smaller feature matrices reduce the value of pre-loading. Its negligible hardware cost for a meaningful vectorized-sparsity speedup makes it one of the more favorable effort-to-gain ratios among the custom vector instructions surveyed.

[19] RISC-V software-support survey. Despite RISC-V's rapid hardware growth, no comprehensive review had analyzed the state of its software support — compilers, operating systems, and security — across

research and industry. The authors conduct a systematic bibliographic survey of five years of literature, classifying the software ecosystem into application fields, hardware platforms, compiler/OS support, and security. They confirm robust support from Linux and FreeRTOS, GCC and LLVM, and QEMU/Gem5/Spike simulators across many hardware platforms, including PULPino, CVA6, SonicBOOM, NOEL-V, and SiFive SoCs. The limitation, as with any literature survey, is that it aggregates existing results rather than presenting new benchmarks, and is limited to the previous five years of publications. Its five-year scope means it necessarily misses the newest generation of Vector-extension AI accelerators and security extensions covered elsewhere in this survey.

[20] RISC-V extensions survey. Despite RISC-V's success in embedded systems, a gap remains between its baseline ISA and the demands of AI, cloud, and cryptography workloads. The authors provide a comprehensive survey categorizing official RISC-V specifications (V, H, and virtual-memory extensions) and academic/industrial custom extensions across eight domains, referencing implementations such as Rocket, BOOM, Xuantie 910, and Vortex GPGPU. They identify trends such as smallFloat instructions for IoT power reduction, convolution/matrix instructions for AI, and modular-reduction instructions for post-quantum cryptography. The limitation acknowledged is that research on extensions for in-memory computing, in-network computing, and scalable trusted computing remains comparatively immature. Its explicit flag on post-quantum cryptography and in-memory computing as immature areas directly informs the gaps we identify in Section IX.

ACKNOWLEDGMENT

The authors thank their course instructor and teaching staff for guidance and feedback during the preparation of this survey.

REFERENCES

- [1] G. Mao, Y. Liu, W. Dai, G. Li, Z. Zhang, A. H. F. Lam, and R. C. C. Cheung, "REALISE-IoT: RISC-V-based efficient and

- lightweight public-key system for IoT applications," *IEEE Internet of Things J.*, vol. 11, no. 2, 2024.
- [2] P. Vizcaino, F. Mantovani, R. Ferrer, and J. Labarta, "Acceleration with long vector architectures: FFT kernel on NEC SX-Aurora and RISC-V vector extension," *Concurrency Computat. Pract. Exper.*, vol. 35, no. 20, e7424, 2023.
- [3] R. Martino, M. Angioli, A. Rosato, M. Barbirotta, A. Cheikh, and M. Olivieri, "Configurable hardware acceleration for hyperdimensional computing extension on RISC-V," *TechRxiv*, Dec. 2024.
- [4] L. Valente et al., "A heterogeneous RISC-V based SoC for secure nano-UAV navigation," *arXiv:2401.03531*, 2024.
- [5] L. Valente, Y. Tortorella, M. Sinigaglia, G. Tagliavini, A. Capotondi, L. Benini, and D. Rossi, "HULK-V: a heterogeneous ultra-low-power Linux capable RISC-V SoC," *arXiv:2211.14944*, 2022.
- [6] N. Sharma, A. Bhatnagar, S. Puri, S. Mitra, and A. Agarwal, "A survey of RISC-V CPU for IoT applications," *SSRN preprint*, 2022.
- [7] S. Razilov, F. Matuš, and G. Fettweis, "Communications signal processing using RISC-V vector extension," in *Proc. IEEE IWCNC*, 2022.
- [8] N. Peccia, F. Haxel, and K. Bringmann, "Tensor program optimization for the RISC-V vector extension using probabilistic programs," *arXiv:2507.01457*, 2025.
- [9] J. van Kempen, T. Jones, D. Mueller-Gritschneider, and U. Schlichtmann, "muRISCV-NN: Challenging Zve32x autovectorization with a TinyML inference library for the RISC-V vector extension," in *Proc. ACM CF '24 Companion*, 2024.
- [10] A. Rezunov, T. Zurstraßen, C. Reimann, and R. Leupers, "Automatic microarchitecture-aware custom instruction design for RISC-V processors," in *Proc. IEEE/ACM ICCAD*, 2025.
- [11] S. Schrammel et al., "SPEAR-V: Secure and practical enclave architecture for RISC-V," in *Proc. ACM ASIA CCS*, 2023, pp. 457–468.
- [12] F. Oppermann et al., "Longnail: HLS of portable custom instruction set extensions for RISC-V from CoreDSL," in *Proc. ACM ASPLOS*, vol. 3, 2024, pp. 591–606.
- [13] K. Balasubramanian et al., "Designing RISC-V instruction set extensions for ANNs: an LLVM compiler-driven perspective," *IEEE Access*, vol. 12, pp. 55925–55944, 2024.
- [14] X. Wang et al., "Logic gate network inference acceleration with RISC-V custom instruction set," in *Proc. ACM CF '25*, 2025, pp. 205–211.
- [15] G. Rutishauser et al., "ColibriES: A milliwatts RISC-V based embedded system with neuromorphic accelerators," *arXiv:2302.07957*, 2023.
- [16] M. Sabih et al., "HW/SW co-design of RISC-V extensions for sparse DNNs," in *Proc. IEEE FPT*, 2024.
- [17] A. Reuther et al., "AI and ML accelerator survey and trends," *arXiv preprint*, 2022.
- [18] N. Titopoulos et al., "IndexMAC: A custom RISC-V vector instruction for structured-sparse matrix multiplications," *arXiv preprint*, 2023.
- [19] M. Mezger et al., "A survey of the RISC-V architecture software support," *IEEE Access*, 2022.

- [20] *Q. Cui et al., "RISC-V instruction set architecture extensions: A survey," IEEE Access, 2023.*